CareLoop trust zones, the authority gate, and the independent read-back

The model proposes. Deterministic code authorises. A separate read of an external system confirms what happened.

UNTRUSTED · CONTROLLED · TRUSTED · work and effects → · independent read-back ⇢ · LLM one of exactly two model-backed stages; neither holds authority

UNTRUSTED what the world sends, and what the models say

Inbound event
an email, its PDF attachments, a portal notice weeks later

TriageResult
`Model.structured_output`
a classification, read as data, never as a rule

Document text
pdfplumber, one page at a time. Instructions inside it stay inert.

PlannerIntent
`one propose_* tool call`
one proposed tool call. A proposal, not an order.

A model's output lands in the same zone as an attacker's document: typed, read, and gated like any other input.

TRUSTED

denied: replan, told a ReasonCode and nothing else

Dispatch — entry, and where a reopened case rejoins the graph

Triage LLM — classify into typed Case fields. Nova Micro

Evidence — read, extract, validate, promote. No model reads it.

Plan LLM — propose exactly one next action. Nova 2 Lite

Gate — execute, ask a human, or send the planner back

Execute — dispatch the tool. Two gates below, one PolicyGate.

Reconcile — re-claim a prepared attempt whose answer never came. Asks no model.

Verify — performed equals authorised, then read the world back

`GraphBuilder · every node subclasses MultiAgentBase · bounds from set_max_node_executions, set_execution_timeout, set_node_timeout`

Caregiver
approve or reject. Releases one exact call. Mints no fact.

PolicyEngine one engine, one rules.yaml, three call sites

ALLOW · DENY · REQUIRE_HUMAN, deterministic, deny by default
rules are declarative in rules.yaml, never encoded in Python
argument classes belong to the tool registry: DESTINATION, AUTHORITY, VALUE
what the model is told back: a closed ReasonCode, never interpolated

`BeforeToolCallEvent.cancel_tool`

`@gated_tool wrapper (backstop)`

Two enforcement sites at the tool boundary, backed by the same deterministic PolicyGate decision — not two engines. Each is proven sufficient alone; with neither, the attack lands.

Application code, the policy engine and rules.yaml. TRUSTED is never manufactured by validation and never by an approval: it comes only from configuration a person wrote.
Case is frozen and transition() is its only writer, so DONE is reachable only from VERIFYING. No graph edge leaves DONE; only an inbound event can.

validated facts, with provenance ↓ · ↑ resolved trust is what the engine reads

CONTROLLED what validation has earned the right to promote, and nothing more

Fact
`{source_doc, page, trust_level}`
Promoted UNTRUSTED to CONTROLLED by validation, and only VALUE-class arguments are promotable at all. DESTINATION and AUTHORITY never are, whatever a validator says.

Case
Frozen. State, evidence, approvals, actions, and an append-only timeline of what was proposed, what was allowed or denied, which rule matched, and what verification observed.

CaseStore
SQLite here, a DynamoDB-shaped interface. The durable source of truth. Strands session persistence holds execution context only and is never asked a business fact.

OUTSIDE CARELOOP

Two external systems supported by the architecture; each is verified by a separate read request.

Demo World clinic
demoworld/ — its own process, its own port. It may never import careloop, which is what makes reading it back evidence rather than the agent consulting itself.

- `POST /clinic/appointments` — the booking, under the idempotency key frozen before dispatch
- `GET /clinic/appointments/{confirmation_id}` — a separate request, compared against what policy authorised

Amazon S3
No idempotency key and no notion of an action, so every guarantee is assembled from create-once primitives. A 412 is settled by fetching and comparing the bytes.

- `PutObject(IfNoneMatch="*")` — upload_document, create-once under that same frozen identity
- `GetObject` — a fresh read; the bytes are digested here, not taken on trust

The other two effects — send_email and grant_portal_access — are simulated, and verification of those closes the loop on the decision, not on the world.

DEPLOYED CareLoop runs as one application process; Demo World runs as a second process on the same EC2 host.

Browser — the dashboard, polling /api/cases

CloudFront — HTTPS. One browser origin, so no CORS

EC2 t4g.small two processes, two ports, two failure domains

- **uvicorn :8000** — the graph above, /api/*, the built frontend
- **demoworld :8100** — /clinic/*, reachable by a judge

Amazon Bedrock — Nova Micro (triage), Nova 2 Lite (planner)

AgentCore Runtime — deployed, invoked, measured: 6.47 s cold, 0.72 s warm. Not on the production path — ADR 0002.

/ and /api/* to :8000 · /clinic/* to :8100

instance profile, no keys on the box

Zone colours are claims about authority, not about code quality: UNTRUSTED means nothing here may decide anything, whoever wrote it.